

Cryptography and Network Security Technical Report

Integrated Situation Assessment

Registration Number: 4202670014

September 29, 2026

1 Risk Assessment

A security review of the polytechnic institute identified critical vulnerabilities across core assets. Table 1 summarizes these assets, vulnerabilities, consequences, risk rankings, and recommended controls.

Asset	Vulnerability	Consequence	Rank	Control
Records Server	Guest access allowed	Data breach / leak	Critical	Firewall / ACL Segmentation
Data in Transit	Unencrypted transfers	Eavesdropping / MitM	High	AES Encryption / SFTP
Staff Accounts	Weak passwords / OS	System compromise	Medium	Password Policy & Patching

Table 1: Risk Assessment Matrix

2 Encryption and Integrity Application

To secure sensitive student records, a Python security toolkit (`crypto_toolkit.py`) was developed using the `cryptography` library.

2.1 Implementation Details

- **Encryption & Decryption:** Implemented via symmetric encryption using Fernet (AES-128 in CBC mode with HMAC authentication). The key is generated and stored outside the repository directory (`../secret.key`).
- **Integrity Checking:** Calculates and verifies SHA-256 hashes before encryption and after decryption to confirm the restored file is identical to the original.
- **Error Handling:** Uses standard Python `try-except` blocks to catch `FileNotFoundError` and `InvalidToken` exceptions gracefully without terminating execution.

2.2 Execution and Test Evidence

Below is the terminal output from running `crypto_toolkit.py`. Note that the output lines starting with `[-]` `Error:...` under Section 4 are **expected test outputs** demonstrating that the error-handling functions successfully capture missing file errors and corrupted decryption attempts without crashing the program:

Listing 1: Python Security Toolkit Execution Output

```
[+] New key generated and saved to '../secret.key'.

--- 1. ENCRYPTION ---
[+] File successfully encrypted and saved to 'sample_student_records.csv.enc'.

--- 2. DECRYPTION ---
[+] File successfully decrypted and saved to 'sample_student_records_decrypted.csv'.

--- 3. INTEGRITY CHECK (SHA-256) ---
Original File Hash: 323
a1d7573ffa2eb128eb3196b892cfde06aa4c7fe52566a9ca6ce3f23c8c122
Expected Hash: 323
a1d7573ffa2eb128eb3196b892cfde06aa4c7fe52566a9ca6ce3f23c8c122
Current Hash: 323
a1d7573ffa2eb128eb3196b892cfde06aa4c7fe52566a9ca6ce3f23c8c122
[+] Integrity Verification Passed: File is untouched.

--- 4. ERROR HANDLING TESTS ---
Testing missing file:
[-] Error: [Errno 2] No such file or directory: 'non_existent_file.enc'

Testing corrupted file decryption:
[-] Error: Decryption failed. Invalid key or corrupted file.
```

3 Network Traffic Filtering

Traffic filtering rules were applied on the host environment using `iptables` to isolate guest networks and protect the server.

3.1 Firewall Rules

```
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp -s 10.0.20.0/24 --dport 22 -j DROP
iptables -A INPUT -p tcp -s 10.0.10.0/24 --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j DROP
```

3.2 Test Results

- **Staff Connection (10.0.10.15):** Permitted (Connection Succeeded).
- **Guest Connection (10.0.20.45):** Blocked (Connection Timed Out).
- **External Connection (172.16.0.88):** Blocked (Connection Timed Out).

4 Conclusion and Submission

The implementation secures student data both at rest (via AES encryption) and in transit (via SHA-256 hashing and host traffic filtering). Network rules effectively isolate unauthorized subnets from sensitive services.

GitHub Repository URL: <https://github.com/cynthianaissa7/cryptography-network-sec>